

RodSki 架构总结：架构设计经验汇报

AI Agent 时代的自动化测试框架，不是把自己做得越来越聪明，而是成为 Agent 稳定可靠的确定性执行底座。

主色调：红 / 黑 / 蓝

主题：架构取舍

关键词：Agent、协议、确定性、活文档

Agent 负责智能，RodSki 负责确定性。

RodSki 的核心定位：不是智能平台，而是确定性执行引擎

Agent

探索、判断、生成、诊断、修复、选择执行范围



XML / SQLite / Plan

可读、可写、可校验、可版本化的活文档协议



RodSki

解析、执行、验证、记录、返回结构化结果

架构不是画图，而是管理取舍

一个长期可靠的落脚点。

放弃

大而全智能测试平台、复杂内置 **Agent** 编排、无限扩展关键字、历史格式过度兼容。

换来

清晰职责边界、稳定协议、可验证执行、更低维护成本和 **Agent** 友好测试资产。

01

AI 的变化很快，执行引擎不能被模型能力和推理策略拖着走。

02

测试执行必须确定，不能依赖自然语言直接猜测动作。

03

TRADE-OFF 02

框架能承载越多 Agent 主动错误和协议滥用的概率越高

架构首先服务业务边界，而不是

技术边界

2

RodSki 把系统拆成两个认知边界：Agent 负责高变化的智能层，RodSki 负责低歧义的执行层。

Agent 负责

- 需求理解与页面探索
- 用例、数据、定位器生成
- 失败诊断与修复策略
- 执行范围选择与报告解读

RodSki 负责

- XML / data / plan 协议校验
- 关键字执行与驱动适配
- 结果、截图、录屏、报告记录
- dry-run / preview / JSON 输出

架构要面向变化，而不是面向当下需求

变化快的部分放外层

- LLM 模型能力
- 页面探索策略
- 视觉识别方案
- 用例生成与失败修复策略

必须稳定的部分沉淀为协议

- XML / SQLite / plan 结构
- 关键字语义和数据引用规则
- CLI 输入输出契约
- dry-run / validate / preview 语义

```
<location type="vision">img/login_btn.png</location>
<location type="ocr">登录</location>
<location type="vision_bbox">100,200,150,250</location>
```

TRADE-OFF 04

架构必须拥有九针羊进簪了

架构必须把复杂性大进龙子

视觉能力是定位能力的增强，不是测试流程语义的扩张。

动作复杂性

通过 `type / send / verify / run` 收敛关键字，而不是无限增加动作。

定位复杂性

关进 `model.xml`，让 case 表达流程，model 表达元素知识。

数据复杂性

关进 `data.sqlite` 和 `globalvalue.xml`，用一致命名减少歧义。

执行复杂性

通过 `plan / selector / debug plan` 隔离执行选择，不污染 case。

特殊复杂性

通过 `run` 脚本承接，不为少数场景污染关键字体系。

TRADE-OFF 05

架构是社会技术系统，不只是代

Agent 时代协议资产模型, 协议资产模型

码系统

放到 Agent 层或可选能力层，核心执行链路不依赖 LLM。

在 Agent 时代，文档不一致不只是影响人读文档，还会直接导致 Agent 生成错误资产。

文档

告诉 Agent 应该如何表达测试资产



Schema / dry-run

限制错误输入，提前发现协议漂移



Demo / Report

证明协议真的可跑，并把结果反馈给 Agent

为什么不是让 AI 直接生成 Playwright 测试代码？

AI 直接生成代码

短期快，但测试意图、定位器、数据、断言、等待策略容易混在脚本里，长期治理成本高。

RODSKI 协议化资产

用 XML / SQLite / plan 分离测试流程、定位知识、测试数据和执行选择，便于 Agent 稳定维护。

直接生成 Playwright 代码：

需求 -> Agent -> Playwright / Pytest 脚本 -> 执行

ARCHITECTURE MODEL

RodSki 的方式：

需求 -> Agent -> XML / SQLite / plan => RodSki -> Playwright / Appium / API / DB -> 结构化结果

RodSki 的分层模型

需求层
直接生成代码解决“快速写出能跑的测试”；RodSki 解决“持续维护可验证、可演进、可审计的测试资产”。
自然语言，适合人和 Agent 沟通

协议层

XML / SQLite / plan, 沉淀测试资产

执行层

RodSki keyword engine, 确定性调度

驱动层

Playwright / Appium / API / DB 执行动作

分析层

Agent 诊断、修复、决策

少

少关键字、少歧义、少等价写法。

最终结论：给 AI Agent 让道，但守住确定性边界

汪

不要做什么

Agent 可生成、可校验、可修复、可演进。

- 不要把 RodSki 做成大而全 Agent 平台
- 不要把自然语言直接作为执行协议
- 不要让关键字、定位器、数据引用无限扩张
- 不要让 AI 任意修改核心测试资产

应该做什么

- 稳定 XML / SQLite / plan 活文档协议
- 强化 dry-run、validate、preview、JSON 输出
- 把变化快的智能能力放到 Agent 层
- 让 RodSki 成为可靠的执行底座

好的 AI 自动化测试架构：需求自然语言表达，测试资产结构化沉淀，执行过程确定可复现，失败结果可诊断，修复动作可验证，

RodSki 项目总结：面向 AI

Agent 的跨平台确定性测试执行引擎

RodSki 的项目定位不是传统自动化脚本库，也不是完整 Agent 平台，而是位于 Agent 与底层自动化工具之间的协议层和执行层。

定位

AI Agent 执行内核 + XML 活文档协议层。

目标

让测试资产可生成、可校验、可执行、可诊断、可演进。

边界

Agent 负责思考，RodSki 负责稳定执行。

CORE CAPABILITIES

RodSki 的核心能力

活文档协议

通过 case / model / data / plan 表达测试流程、元素定位、测试数据和执行选择。
-> RodSki 确定性执行

-> JSON / report / screenshot / recording

-> Agent 分析与修复

跨平台执行

支持 Web、Android、iOS、Desktop、API、DB 等测试对象，上层语义尽量统一。

核心关键字

以 `type / send / verify / run` 收敛测试动作，降低 Agent 生成和维护成本。

数据驱动

RodSki 核心架构：协议解析、关键字执行、驱动适配、结果输出

视觉定位

支持传统定位器、OCR、图片匹配、坐标等方式，视觉能力作为 locator type 扩展。

Agent 友好 CLI CLI Layer

提供 run、dry-run、explain、data、init、plan 等命令，输出结构化结果供 Agent 消费。
rodski run / data / init / explain / dry-run / plan

核心能力不是“功能越多越好”，而是↓协议稳定、执行确定、反馈可解析”。

Parser Layer

CaseParser、ModelParser、DataParser、PlanParser



Core Engine

TaskExecutor、KeywordEngine、ResultWriter、配置与日志

Driver Layer

Playwright、Appium、PyWinAuto、REST、DB Driver

AGENT ARCHITECTURE

rodski-agent 架构↓ 设计、执行、诊断、修复的智能层

智能等待、视觉定位、录屏、诊断、报告、观测

Design Agent

需求分析、页面探索、元素识别、用例规划、数据设计、XML 生成



Output Layer

JSON summary、XML report、log、screenshot、recording



Execution Agent

预检查点 调用 rodski run、解析结果、判断失败类型
设计点

上层协议稳定，底层驱动可替换。



避免问题

Repair / Diagnose

定位失败、数据失败、断言失败、等待问题，生成修复建议或补丁

Agent 层做什么

- 从自然语言需求生成测试资产
- 通过页面探索生成 model 和数据
- 根据执行结果诊断失败原因
- 在受控边界内修复资产并重试

Agent 层不做什么

- 不替代 RodSki 的确定性执行

而只基础数据信自

项目全生命周期

- 不绕过 XML / data / plan 契约
- 不把 LLM 推理混入核心执行结果

技术栈

Python 3.9+, Playwright, Appium, PyWinAuto, SQLite, Click, LangGraph, LangChain provider。

```
rodski-agent design -> generate XML / SQLite / plan
```

```
rodski-agent run -> call rodski -> parse JSON result
```

```
rodski-agent pipeline -> design + validate + execute + diagnose
```

```
rodski-agent diagnose -> failure analysis + repair direction
```

`rodski/` 执行引擎，`rodski-agent/` 智能层，`rodski-demo/` 示例，`rodski/docs/` 文档。

测试资产

`case/\*.xml`、`model/model.xml`、`data/data.sqlite`、`data/globalvalue.xml`、`plan/\*.xml`。

输出资产

JSON 结果、XML 报告、执行日志、截图、录屏、Return 值、skipped 原因。

安装入口

`pip install -e .`、`pip install -e "[web]"`、`pip install -e rodski-agent/`。

PROJECT SUMMARY

项目总结：PodSki 的价值呈现

项目心得：Rodski 的闭环走向

AI 自动化测试从“脚本生成”走向“资产治理”

RodSki 解决的问题

- AI 生成测试代码难以长期治理
- 多平台测试语义分裂
- 测试数据、定位器、断言和流程耦合
- 失败结果难以被 Agent 结构化理解
- 测试资产缺少稳定协议和校验边界

RodSki 形成的能力闭环

- Agent 生成结构化测试资产
- RodSki 校验并确定性执行
- 结果结构化返回给 Agent
- Agent 基于失败原因修复资产
- 测试资产持续沉淀和演进

RodSki 的最终价值：把 AI 的创造力放在测试设计和修复上，把执行确定性、协议一致性和资产可治理性留在框架内。